Lista de exercícios 4 – Pilha Dinâmica

1. Desenvolva uma função para retornar a média aritmética dos valores armazenados na pilha.

   Observação: Percorrer os elementos da pilha usando um laço de repetição, acessando o primeiro elemento pelo ponteiro "topo" e os demais elementos com o ponteiro "prox", semelhante a lógica usada para implementar a função *mostraP*.

2. Empregando pilha, implemente um programa para inverter a ordem das letras nas palavras em uma frase. Por exemplo, na frase "teste de pilha", o programa deverá exibir "etset ed ahlip".

   Observação: É proibido o uso de vetores, use uma pilha dinâmica para o armazenamento de dados.

3. Empregando pilha, construa um programa para ordenar uma sequência de valores inteiros <u>fornecidos pelo usuário</u>. Faça uso de duas pilhas: ORD e AUX. O objetivo é organizar a pilha ORD de modo que nenhum item seja empilhado sobre outro menor, fazendo uso da pilha auxiliar AUX.

   a) O usuário pode informar quantos valores inteiros ele desejar.

   b) Ao final, o programa deve mostrar a pilha ORD.

4. Empregando pilha, crie uma função para verificar se uma expressão composta apenas por chaves, colchetes e parênteses está ou não *balanceada*.

   a) Exemplo de expressões balanceadas: "`[ { () () } {} ]`", "`{ [ ( [ { } ] ) ] }`"

   b) Exemplo de expressões não balanceadas: "`{ [ ( } ] )`", "`{ [ ) () ( ] }`"

5. Crie uma função para copiar os dados pilha P1 para P2. Esta função deve <u>preservar os dados armazenados em P1</u>, ou seja, após a execução da função, P1 e P2 devem possuir os mesmos elementos e na mesma ordem.

6. O banco BanPasso possui diversos clientes. O cadastro de novos clientes do banco é realizado perante o preenchimento de uma ficha, a qual possui os seguintes dados: **código, nome, idade, saldo em conta** (valor do depósito inicial). Com a ficha de um cliente preenchida, o cadastro é concluído empilhando na pilha CADASTRO. Não pode haver clientes com o mesmo código cadastrado e a ficha é empilhada sempre mantendo a ordem alfabética dos clientes como no exemplo baixo:

TOPO

Ana, código 1, idade 20, saldo 3000
Nivaldo, código 2, idade 15, saldo 1000
Otaviano, código 3, idade 16, saldo 2000

BASE

Se a cliente Maria for cadastrada na sequência, a pilha ficará da seguinte forma:

TOPO

Ana, código 1, idade 20, saldo 3000
Maria, código 4, idade 60, saldo 20000
Nivaldo, código 2, idade 15, saldo 1000
Otaviano, código 3, idade 16, saldo 2000

BASE

Observações:

- Utilize a estrutura de dados pilha dinâmica na implementação;

- Não é permitido o uso de vetores ou matrizes, use pilhas auxiliares para o armazenamento de dados.

- Para verificar a ordem alfabética entre duas strings, basta usar o operador ">" ou "<", exemplo:

```
string nome1="maria", nome2="ana", nomeAux;
```

```
if(nome2<nome1)
{
      //vai entrar neste IF
      nomeAux = nome1;
      nome1 = nome2;
      nome2 = nomeAux;
}
```

7. Crie uma função recursiva para imprimir os elementos da pilha na ordem inversa.

8. Um intervalo de valores de uma pilha P é definido por dois números inteiros **I** (início) e **F** (fim), que são os elementos da pilha que estão entre as posições **I** e **F**, sentido topo para base. Considerando duas pilhas dinâmicas $\mathbf{P_1}$ e $\mathbf{P_2}$, implemente uma função em C++ para verificar se um intervalo de valores de $\mathbf{P_1}$ é igual ao inverso de um intervalo de valores de $\mathbf{P_2}$.

   Exemplo:

   P_1: topo [-1, **20, 10,** 5] base
   Intervalo de P_1: I_1=1 e F_1=2

   P_2: topo [**10, 20,** -1, 5] base
   Intervalo de P_2: I_2=0 e F_2=1

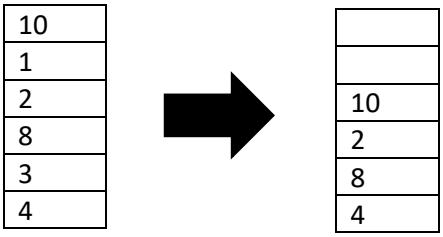
   Intervalos P_1[1-2] e P_2[0-1] inversos? **Sim**

   O protótipo da função deve ser:
   `bool` **`verificaIntervalosInversos`**`(Pilha *p1, Pilha *p2, int i1, int f1, int i2, int f2);`

   O programa deve exibir na tela:

   - Imprimir as pilhas $\mathbf{P_1}$ e $\mathbf{P_2}$ antes da chamada da função `verificaIntervalosInversos`.
   - Imprimir os intervalos de ambas as pilhas e "sim" se o retorno da função `verificaIntervalosInversos` for verdadeiro, ou "não" caso contrário.
   - Imprimir as pilhas $\mathbf{P_1}$ e $\mathbf{P_2}$ depois da chamada da função `verificaIntervalosInversos`.

   Observações:

   - Utilizar a estrutura de dados Pilha dinâmica na implementação.
   - É permitido o uso de pilha auxiliares.
   - As pilhas $\mathbf{P_1}$ e $\mathbf{P_2}$ devem permanecer inalteradas, ou seja, os elementos de $\mathbf{P_1}$ e $\mathbf{P_2}$ devem ser os mesmos e na mesma ordem, antes e depois da chamada da função `verificaIntervalosInversos`.
   - A função `verificaIntervalosInversos` deve retornar "*false*" se um intervalo for inválido, conforme os seguintes casos:
     - (**I**<0)
     - (**F**<0)
     - (**F**<**I**)
     - (**F**>=**T**) ,sendo **T** o total de elementos da pilha.

9. Crie uma função para remover todos os elementos **ímpares** de uma pilha P, mantendo a mesma ordem relativa dos elementos remanescentes. Exemplo: